

```

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score, precision_score, recall_score

# Load dataset
df = pd.read_csv("617ec7a0-b7f1-423e-b810-23f59803ffb6.csv")

# Display column names to see what we're working with
print("Column names:")
print(df.columns.tolist())

# Handle missing values
if 'Arrival Delay in Minutes' in df.columns:
    df['Arrival Delay in Minutes'].fillna(df['Arrival Delay in Minutes'].median(), inplace=True)

# Drop unnecessary columns if they exist
columns_to_drop = []
for col in ['Unnamed: 0', 'id', 'ID']:
    if col in df.columns:
        columns_to_drop.append(col)
if columns_to_drop:
    df = df.drop(columns_to_drop, axis=1)

# Feature Engineering - Service Quality Average
service_cols = []
service_candidates = ['Seat comfort', 'Inflight wifi service', 'Inflight entertainment',
                      'Online support', 'Ease of Online booking', 'On-board service',
                      'Leg room service', 'Baggage handling', 'Checkin service',
                      'Cleanliness', 'Online boarding']

for col in service_candidates:
    if col in df.columns:
        service_cols.append(col)

if service_cols:
    df['Service_Quality_Avg'] = df[service_cols].mean(axis=1)

# Feature Engineering - Total Delay
if 'Departure Delay in Minutes' in df.columns and 'Arrival Delay in Minutes' in df.columns:
    df['Total Delay'] = df['Departure Delay in Minutes'] + df['Arrival Delay in Minutes']
    df['Had_Delay'] = (df['Total Delay'] > 0).astype(int)

# Identify categorical columns
categorical_cols = df.select_dtypes(include=['object']).columns.tolist()
categorical_cols = [col for col in categorical_cols if col != 'satisfaction']

print(f"\nCategorical columns found: {categorical_cols}")

# Encode categorical variables
label_encoders = {}
for col in categorical_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col].astype(str))
    label_encoders[col] = le

# Encode target variable
target_encoder = LabelEncoder()
df['satisfaction_encoded'] = target_encoder.fit_transform(df['satisfaction'])

```

```

df['satisfaction_encoded'] = target_encoder.fit_transform(df['satisfaction'])
print(f"\nTarget encoding: {dict(zip(target_encoder.classes_, target_encoder.transform(target_encoder.classes_)))}"

# Define features - only use columns that exist
feature_candidates = ['Customer Type', 'Age', 'Type of Travel', 'Class', 'Flight Distance',
                      'Seat comfort', 'Departure/Arrival time convenient', 'Food and drink',
                      'Gate location', 'Inflight wifi service', 'Inflight entertainment',
                      'Online support', 'Ease of Online booking', 'On-board service',
                      'Leg room service', 'Baggage handling', 'Checkin service',
                      'Cleanliness', 'Online boarding']

# Add engineered features if they exist
additional_features = []
for feat in ['Total Delay', 'Had_Delay', 'Service_Quality_Avg']:
    if feat in df.columns:
        additional_features.append(feat)

# Only use features that actually exist in the dataframe
features = []
for col in feature_candidates + additional_features:
    if col in df.columns:
        features.append(col)

print(f"\nFeatures being used: {features}")

X = df[features]
y = df['satisfaction_encoded']

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"\nTraining set: {X_train.shape[0]} rows")
print(f"Test set: {X_test.shape[0]} rows")

# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train Logistic Regression
log_reg = LogisticRegression(
    C=1.0,
    solver='lbfgs',
    max_iter=1000,
    class_weight='balanced',
    random_state=42
)
log_reg.fit(X_train_scaled, y_train)

# Predictions
y_pred = log_reg.predict(X_test_scaled)
y_pred_proba = log_reg.predict_proba(X_test_scaled)[: , 1]

# Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_pred_proba)

print("\n" + "="*50)
print("MODEL PERFORMANCE METRICS")
print("="*50)
print(f"Accuracy: {accuracy:.4f}")

```

```

print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"ROC-AUC: {roc_auc:.4f}")
print("="*50)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)

# Visualize Confusion Matrix
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Dissatisfied', 'Satisfied'],
            yticklabels=['Dissatisfied', 'Satisfied'])
plt.title('Confusion Matrix - Logistic Regression')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=['Dissatisfied', 'Satisfied']))

# ROC Curve
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'Logistic Regression (AUC = {roc_auc:.3f})', linewidth=2)
plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier', linewidth=1)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve - Airline Satisfaction Prediction')
plt.legend()
plt.grid(alpha=0.3)
plt.show()

# Feature Importance
coefficients = log_reg.coef_[0]
coef_df = pd.DataFrame({
    'Feature': features,
    'Coefficient': coefficients,
    'Odds_Ratio': np.exp(coefficients)
})
coef_df['Abs_Coefficient'] = np.abs(coef_df['Coefficient'])
coef_df = coef_df.sort_values('Abs_Coefficient', ascending=False)

print("\n" + "="*50)
print("TOP 10 FEATURES INFLUENCING SATISFACTION")
print("="*50)
print(coef_df[['Feature', 'Coefficient', 'Odds_Ratio']].head(10).to_string(index=False))

# Visualize top coefficients
plt.figure(figsize=(10, 8))
top_features = coef_df.head(15)
colors = ['green' if x > 0 else 'red' for x in top_features['Coefficient']]
plt.barh(top_features['Feature'], top_features['Coefficient'], color=colors)
plt.xlabel('Coefficient Value')
plt.title('Top 15 Features Influencing Satisfaction')
plt.axvline(x=0, color='black', linestyle='--', alpha=0.5)
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()

# Summary
print("\n" + "="*50)

```

```
print("SUMMARY")
print("="*50)
print(f"Dataset: {df.shape[0]} rows, {df.shape[1]} columns")
print(f"Features used: {len(features)}")
print(f"Model Accuracy: {accuracy:.4f} ({accuracy*100:.1f}%)")
print(f"ROC-AUC: {roc_auc:.4f}")
print("\nTop 3 Positive Drivers:")
for i, (_, row) in enumerate(coef_df[coef_df['Coefficient'] > 0].head(3).iterrows(), 1):
    print(f"  {i}. {row['Feature']}: {row['Coefficient']:.4f}")
print("\nTop 3 Negative Drivers:")
for i, (_, row) in enumerate(coef_df[coef_df['Coefficient'] < 0].head(3).iterrows(), 1):
    print(f"  {i}. {row['Feature']}: {row['Coefficient']:.4f}")
```



```
Column names:
['satisfaction', 'Customer Type', 'Age', 'Type of Travel', 'Class', 'Flight Distance', 'Seat comfort', '

Categorical columns found: ['Customer Type', 'Type of Travel', 'Class']
/tmp/ipykernel_13455/4287676328.py:19: FutureWarning: A value is trying to be set on a copy of a DataFra
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate obj

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplac
```

```
df['Arrival Delay in Minutes'].fillna(df['Arrival Delay in Minutes'].median(), inplace=True)
```

```
Target encoding: {'dissatisfied': np.int64(0), 'satisfied': np.int64(1)}
```

```
Features being used: ['Customer Type', 'Age', 'Type of Travel', 'Class', 'Flight Distance', 'Seat comfor
```

```
Training set: 103904 rows
```

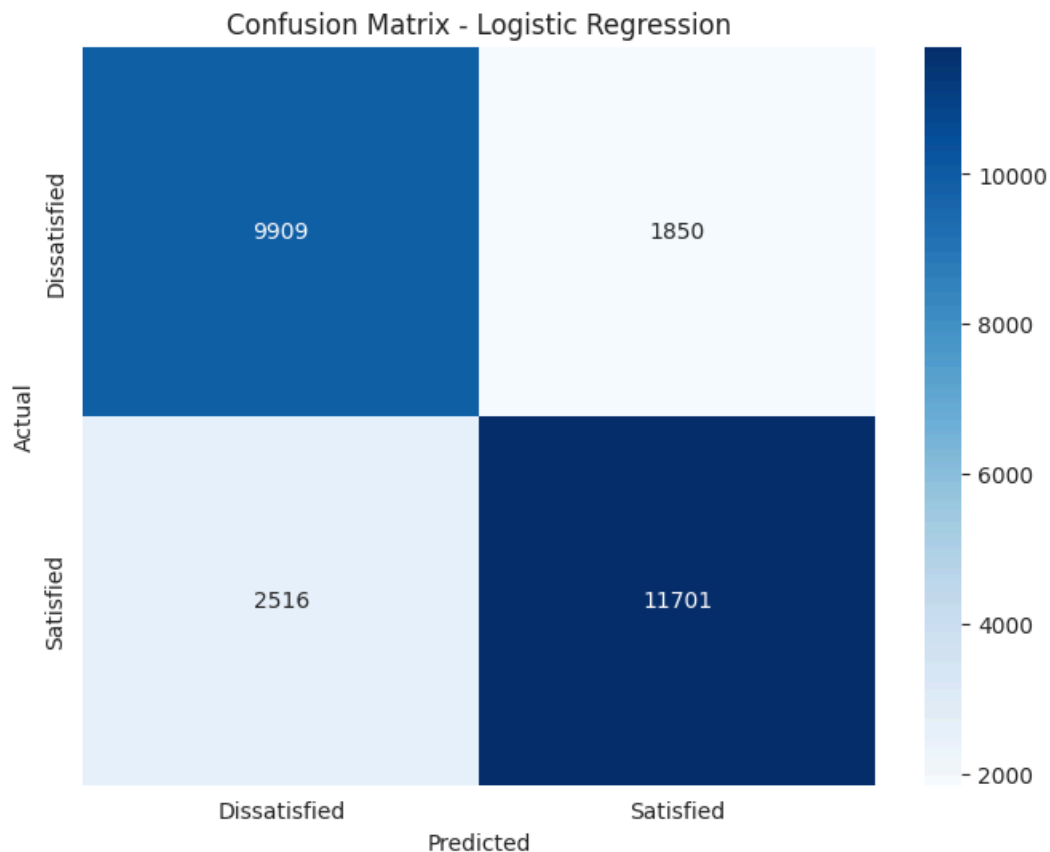
```
Test set: 25976 rows
```

```
=====
MODEL PERFORMANCE METRICS
=====
Accuracy:  0.8319
Precision:  0.8635
Recall:     0.8230
ROC-AUC:    0.9032
=====
```

```
Confusion Matrix:
```

```
[[ 9909 1850]
```

```
 [ 2516 11701]]
```



```
Classification Report:
```

	precision	recall	f1-score	support
Dissatisfied	0.80	0.84	0.82	11759
Satisfied	0.86	0.82	0.84	14217
accuracy			0.83	25976